PIR: A Prompt Intermediate Representation for Parsing, Analysing, Re-stringing and Refactoring LLM Prompts

Ashish Ranjan Sharma

Technical report • Version 2.4 • October 2026

Abstract

Production prompts grow like legacy code. Rules are appended rather than integrated. Conditions are scattered through Jinja templates. Duplicates and contradictions accumulate, and per-request values leak into the opening lines, where they defeat prompt caching. We propose *Prompt Anatomy*, which treats a prompt as a program and compiles it into a *Prompt Intermediate Representation* (PIR): typed, colour-coded statements ("pearls") with source spans, guards and dependency edges. The front end uses tree-sitter to locate prompts in code and recover their markdown structure, and Jinja's own syntax tree to recover conditional guards. A semantic layer uses embeddings to propose candidates (duplicates, topic clusters, conflict pairs, intent membership) and an LLM to judge only those candidates. The LLM returns edit operations on the PIR, never free text. A back end re-strings the pearls: it first proves a byte-identical round trip, then reorders under dependency and caching constraints and checks that every obligation survives. For templates, we apply compiler-style refactorings (deduplication, hoisting, dead-branch removal, block grouping, if/else consolidation) and verify them by *translation validation* over every flag combination. On a worked banking example, the pipeline rebuilds the prompt byte-for-byte, finds an account-number conflict and a cache-breaking variable, and refactors a template with interleaved conditions into grouped blocks with zero mismatches across all variants. One approved rewrite raises the cacheable prefix from 8 to 79 words per variant. Version 2.0 adds two structural ideas. The *block stack* breaks a prompt into independently movable blocks, optimises each with its own LLM task, and merges them back recursively. The *Prompt Stack* borrows encapsulation from the OSI model: pearls are placed on a grid of concern layers and scope envelopes, so ordering becomes a lookup, the cacheable prefix is maximal by construction, and layer violations can be linted. On the worked template, the Prompt Stack output is 9.7% shorter than the original, doubles the cacheable prefix, and passes validation with zero mismatches. Version 2.1 evaluates the passes on 19 public Jinja prompt templates from four open-source agent projects: 1,136 segments and about 22,000 tokens, with zero validation mismatches across 2,353 variants. It shows that structural recognition (section tags, code blocks, example transcripts) must precede deduplication, that duplication in curated prompts lives between files rather than within them, and that moving per-request values inward can raise the cacheable prefix from 40 to 845 tokens. Version 2.2 applies the same separation of concerns one level up, across agents. A prompt is split into agents along the cut that minimises expected input per request: identity and policy are copied, procedures and tools are partitioned by intent, and an edge that crosses the cut becomes a copied anchor or a handoff contract. Agents are merged by aligning their pearls, subsuming duplicates into one global envelope, flagging conflicts, and guarding each agent's procedures, with translation validation per mode. On a worked banking prompt, the cut lowers expected static input from 1,033 to 676 tokens per request at the cost of one router call, and merging the two agents back subsumes seven pearls with zero mismatches. Version 2.3 adds a *PIR table*, a versioned codebook of definitions, cues, counter-examples and tie-breaks that the LLM receives as a cached prefix when it tags statements the rules are unsure of, and a 50-page synthetic prompt with gold labels. On it the rule layer is confident on 93% of 2,126 statements at 99.0% accuracy and routes the rest, including 39 of 42 deliberately ambiguous sentences on which rules alone are right only 28.6% of the time; the automatic passes cut tokens per variant by 10.3% and raise the cacheable prefix from 18 to about 28,000 tokens, with zero mismatches across 1,536 variants. Version 2.4 treats the prompt the way refactoring tools treat code, as a *prompt property graph*: the template and markdown trees plus def-use edges for tools and defined terms and anaphora edges. Slicing that graph gives intent packs that are 16% smaller than section-based packs and leave no dependency behind, where section packs miss 208. Every component runs from a short Python script, so it can serve as a tool for a Copilot agent in VS Code.

1 Introduction

Prompts in enterprise agent systems are long-lived artefacts. They are edited by many people, assembled from templates and tuned against incidents. Their failure modes resemble those of unmaintained code:

- duplicated rules;
- contradictory rules;
- conditions interleaved with unconditional text;
- dead branches;
- instructions whose meaning depends on a neighbour that has since moved, such as "Otherwise, . . . ".

Unlike code, prompts have no compiler, no type system and no refactoring tools. Teams therefore either live with the bloat or compress it with token-level methods [10], which shorten text without understanding its structure.

The cost is measurable. Instruction-following accuracy falls as the number of simultaneous instructions grows [2]. Long contexts degrade even with perfect retrieval [3, 4], and information in the middle is used least [1]. OpenAI's guidance for GPT-5.6 warns that "conflicting rules can create more instability than missing detail" [5]. Prompt caching rewards a stable prefix and punishes per-request values near the top [6, 7].

We take the compiler view literally. A prompt is parsed into an intermediate representation, analysed by passes, transformed by semantics-preserving refactorings, and emitted again with a proof obligation. The metaphor that guided the design is a *broken garland*. Splitting a prompt into statements is easy, like picking up pearls from the floor. What is lost is the thread: order, scope, dependencies and conditions. PIR keeps the thread.

Contributions.

(1) A statement-level IR for prompts, with a kind taxonomy and colour map, source spans, guards and dependency edges (§3).

(2) A front end combining tree-sitter (code and markdown) with the Jinja AST (§4).

(3) A division of labour between embeddings, which propose candidates, and an LLM, which judges them and returns PIR edit operations, guided by a versioned *PIR table* (§5).

(4) A lossless re-stringing algorithm with structural proof checks (§6).

(5) Compiler-style refactoring of conditional templates, verified by translation validation over all variants (§7).

(6) A *block stack* that divides a prompt into independently movable blocks, optimises them in parallel and merges them recursively (§8).

(7) The *Prompt Stack*, an OSI-inspired layering with concern layers, scope envelopes, outward-only dependencies and lint rules (§9).

(8) *Separation of concerns across agents*: splitting a prompt into agents along a cost-minimising cut, and merging agents into one guarded template, each with checks (§10).

(9) An evaluation on 19 public prompt templates, and a canvas-based explorer for prompts with hundreds of segments (§12, §13).

(10) A worked case study and an integration path for Copilot in VS Code (§11, §14).

2 Background

Prompts as software. Context-engineering guidance asks for "the smallest set of high-signal tokens" and warns against overlapping tools and rules [9]. Manus reports that the KV-cache hit rate is the single most important production metric for agents. It recommends a stable, append-only prefix [8]. Prompt optimisers such as DSPy and GEPA search over prompt text against a metric [11, 12]. They are complementary to PIR, which gives them a structured search space and a safety net.

Compression versus refactoring. LLMLingua-style compressors drop tokens by estimated importance [10]. They do not know that "Otherwise" binds to the previous rule, or that a sentence is guarded by `{% if is_premium %}`. Compressing the static prefix on every request also breaks caching. PIR works one level up: it changes *which statements exist, where they sit and under which conditions*, and it proves that nothing required was lost.

Compiler techniques. We borrow established ideas:

- **Parsing into a syntax tree** with tree-sitter [13]. It is error-tolerant, incremental, multi-language and gives byte spans.
- **Path conditions** for guarded code.
- **Partial evaluation** over a finite flag domain [15].
- **The refactoring catalogue** [16].
- **Translation validation** [25], which checks each transformed program rather than proving the transformer correct once.

Layered architectures. The OSI reference model separates communication into layers with fixed interfaces. Each layer encapsulates the data of the layer above with its own header, and peer layers follow their own protocols [17, 18]. Section 9 transfers these principles to prompts.

Separation of concerns. Dijkstra named the principle of studying one aspect of a problem in isolation [26], and Parnas showed that modules should be cut along design decisions likely to change, not along processing steps [27]. Section 10 applies both ideas to the question of which concerns belong in one agent.

3 The Prompt Intermediate Representation

A PIR document is a list of *pearls* plus a set of edges. Each pearl is one statement, usually a sentence or a list item, carrying the fields in Table 1. Table 2 gives the kind taxonomy and the colour map used in the Prompt Anatomy views.

Table 1: Fields of a PIR pearl.

Field	Meaning
`id`, `text`	Stable identifier and statement text.
`kind`	Role, condition, instruction, prohibition, format, tool, example or context (Table 2).
`modality`	*must*, *should*, *may* or none; prohibitions count as *must*.
`section`	Heading path, e.g. `Rules / Transfers`.
`guard`	Conjunction of template conditions under which the pearl prints (its path condition).
`vars`, `dynamic`	Template variables used; *dynamic* if any value changes per request.
`span`, `trivia`	Byte range in the source and the exact text between the pearl and its predecessor (whitespace, list marker, heading). These enable a lossless round trip.
`deps`	Ids this pearl leans on: anaphora, exception-of, example-of, step-order.

Edges. PIR records five edge types:

- *sequence*: original order;
- *scope*: section or XML-tag containment;
- *depends-on*: anaphora such as “Otherwise”, “this” or “the above”, detected lexically;
- *exception-of* and *example-of*: proposed by the LLM;
- *guarded-by*: the template condition.

Edges are what make re-stringing safe: a pearl never travels without the pearls it leans on. Figure 1 shows the Prompt Anatomy view of a small banking prompt as classified by the rule layer.

Table 2: Kind taxonomy, detection cue and colour.

Kind	Typical cue (rule layer)	Colour
role	“You are”, “Act as”	blue
condition	“If”, “When”, “Unless”, “Only if”	orange
instruction	Imperative verb; “Always”, “Must”, “Ensure”	green
prohibition	“Never”, “Do not”, “Must not”, “Avoid”	red
format	“Respond in”, “JSON”, “bullets”, “length”	violet
tool	“call / use / invoke” plus “tool / function / API”	aqua
example	“For example”, “e.g.”	–
context	No cue matched; sent to the LLM classifier	grey
dynamic (overlay)	Contains a per-request `{{variable}}`	amber outline

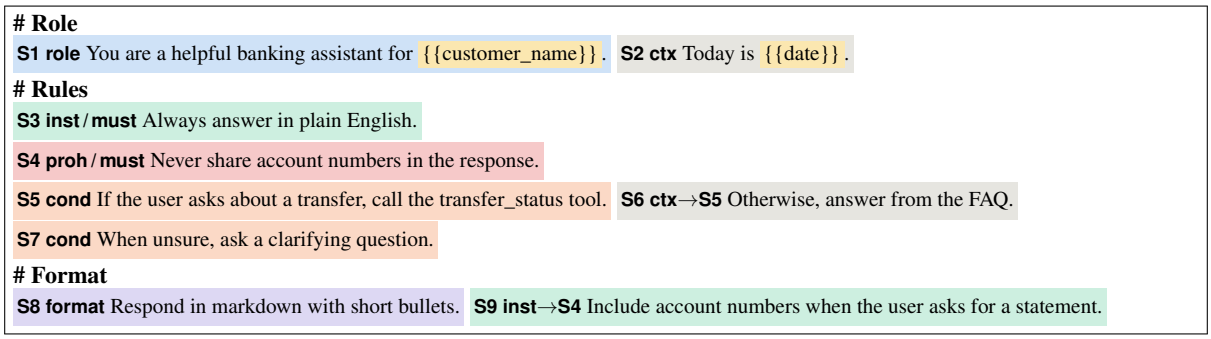


Figure 1: Prompt Anatomy view of the sample prompt. The labels show pearl id, kind and modality; arrows mark dependency edges (S6 leans on S5; the LLM judged S9 an exception to S4). Amber marks per-request variables: two sit in the first two pearls, so the cacheable prefix ends almost immediately.

4 Front End: Finding and Parsing Prompts

Locating prompts in code. Prompts live inside application code, for example as `instructions=` or `system_prompt=` keyword arguments to Microsoft Agent Framework `Agent(...)` calls. A tree-sitter query over `tree-sitter-python` finds these arguments and their line numbers. Python’s `ast` module could do the same for Python alone. Tree-sitter adds grammars for C#, YAML and JSON configuration, tolerates half-edited files, and returns byte spans for in-place edits.

Structure. Prompt bodies are mostly markdown. `tree-sitter-markdown` yields the heading hierarchy, list items and paragraphs. We walk the tree, maintain the section path, split paragraphs into sentences and record each sentence’s byte span. The prototype walks nodes directly rather than using the query API, which changed across recent py-tree-sitter releases. It installs from prebuilt wheels, so no compiler is needed on locked-down laptops.

Templates. For Jinja templates, tree-sitter is the wrong tool: Jinja already exposes its syntax tree through `Environment.parse`. We walk `If`, `elif` and `else` nodes and assign each text fragment the conjunction of enclosing tests. An `elif` carries the negation of earlier tests. Fragments are then joined into lines. A line whose fragments share one guard becomes a guarded pearl. A line that is only an inline `if`/`else` is split into one pearl per branch. A line whose condition sits mid-sentence is kept intact and flagged as *inline*, so it is never silently restructured.

Rule classification. An ordered list of lexical cues (Table 2) assigns kind and modality. Rules are fast, deterministic and auditable. Statements that match no cue fall through to the LLM (§5).

5 Semantic Layer: Embeddings Propose, the LLM Judges

Embeddings and LLMs have different cost and reliability profiles, so PIR gives them different jobs (Table 3). Embeddings run once per prompt, cheaply and repeatably, and turn an $O(n^2)$ comparison space into a handful of candidates. The LLM sees only those candidates and returns JSON that conforms to a schema.

Table 3: Division of labour in the semantic layer.

Pass	Embeddings (propose)	LLM (judge, return PIR ops)
Near-duplicates	Cosine similarity above a calibrated threshold	Confirm and `merge`
Topic clusters	Agglomerative clustering on cosine distance	`compress_cluster`: fewest statements that keep every must/never
Conflicts	Same topic, opposite polarity (prohibition vs. instruction)	Contradiction / exception / duplicate / compatible, plus ops (e.g. `add_condition`)
Intent membership	Similarity to centroids of example user turns per intent	— (used to build intent packs; rules serving no intent are dead-weight candidates)
Unclassified pearls	—	Normalise to {kind, modality, when, action, object}

Operations, not prose. The LLM never rewrites the prompt. It returns operations such as `merge`, `delete`, `rewrite`, `add_condition` or `move`, addressed by pearl id. The back end applies them deterministically, so every change is attributable, reviewable in a diff, and reversible.

Practicalities.

- **Thresholds.** They depend on the embedding model; calibrate them once on about 50 labelled pairs.
- **Caching.** Cache results by a hash of the pearl text, so a small edit re-processes only what changed.
- **No per-request retrieval.** Do not retrieve rules by embedding at request time: it changes the prefix on every call and defeats caching. Use intent membership offline to build stable per-intent packs instead.
- **No API access.** Where the analysis cannot call an API, it writes the LLM tasks to a JSON file for a Copilot agent to judge (§14).

The PIR table: a codebook for the LLM. Embedding quality decides which candidates the LLM ever sees; the instructions the LLM tags with decide how consistently it labels them. A one-line request ("classify this sentence") leaves every ambiguous case to the model's guess. PIR therefore ships a *PIR table* (`pir_table_v1.json`): for each field (kind, modality, layer, envelope, dependency, intent) it lists every value with a definition, detection cues, a positive example, *not-this* counter-examples and a tie-break rule, plus the output schema. Table 4 shows an excerpt. Three design choices keep it practical. First, the rule layer stays in front: it tags what it is confident about and routes only the rest. A sentence is routed when no leading cue matched, when it contains a hedge (*possible, allowed, try, can, please, only, …*), or when cues conflict. Second, the table never changes between calls, so it sits in the cacheable prefix and only the batch of routed statements is new, which is the Prompt Stack (§9) applied to PIR's own tagging prompt. Third, every tag returns its evidence span and a confidence, and low-confidence tags go to a person.

Measuring the tagging. The 50-page synthetic prompt of §12 is generated with a gold label for every statement, and 42 of its 2,126 statements are drawn from 24 deliberately ambiguous phrasings taken from the table's not-this and tie-break rows. Table 5 scores the rule layer and its routing. Rules are confident on 1,968 statements (93%) and right on 99.0% of them; the 1.0% they get wrong are mostly "This section covers …" lines that a cue misreads as an instruction. The 158 routed statements contain 39 of the 42 ambiguous ones, on which rules alone are right only 28.6% of the time. That is the slice the PIR table is written for: four LLM calls of 40 statements each, with the table cached. The harness (`pir_tag.py`) scores a thin prompt and the table on the routed statements with any OpenAI-compatible endpoint; we report those numbers only once they are run on the deployment's own model. The hedge list was written alongside the ambiguous pool, so routing recall here is optimistic and must be re-measured on real prompts.

Table 4: Excerpt of the PIR table (`kind` field). The full table also covers modality, layer, envelope, dependency and intent, and fixes the JSON output schema.

Tag	Cues	Not this	Tie-break
role	You are, Act as	"You are allowed to waive one fee" → instruction	Only the first identity statement
prohibition	never, do not, must not, avoid, under no circumstances	"Do not hesitate to ask" → instruction; "Never mind the old format" → context; "Do not use tables" → format	A forbidden action wins over a condition in the same sentence; the condition goes to `when`
condition	if, when, unless, only if, only after	"If possible, keep it short" → format (a hedge)	Needs a situation that can be true or false at run time
instruction	imperative verb, always, must, make sure	"The monthly fee is 4.95 dollars" → context	Default for imperatives not about the reply's form
format	respond in, reply in, JSON, bullets, under *N* words	"Explain the fee in one sentence" → instruction	Constrains only how the reply looks or sounds
tool	use / call / invoke + tool name	"Tell the customer to use the app" → instruction	Must name a tool from the tool list
example	Example, For example, e.g.	–	Never deduplicated
context	no imperative	–	Fallback

Table 5: Rule-layer tagging on the 50-page synthetic prompt, scored against gold labels (accuracy, %).

Subset	Statements	Kind	Modality
All	2,126	97.3	97.4
Rules confident	1,968	99.0	98.8
Routed to the LLM	158	75.9	79.1
Ambiguous phrasings (39 of 42 routed)	42	28.6	28.6
Routed, LLM with thin prompt	158	to be run (`pir_tag.py --llm thin`)	
Routed, LLM with PIR table	158	to be run (`pir_tag.py --llm table`)	

Algorithm 1 Constrained, stable re-stringing

Require: pearls P with `pos`, `section`, `kind`, `dynamic`, `deps`

for $p \in P$ **do** ▷ a dependant lives where its anchor lives
  $a \leftarrow p$
  while $a.deps \neq \emptyset$ **do** $a \leftarrow$ anchor of a
  end while
  $home(p) \leftarrow a.section$ **if** $a.kind = role$ **else** (CONTEXT **if** $a.dynamic$ **else** $a.section$)
end for
order sections by first appearance; move CONTEXT last ▷ static before dynamic
for each section s **do**
  for each root r in s by `pos` **do**
    emit r; then emit its dependants recursively, by `pos`
  end for
end for
check: obligations kept; no dependant precedes its anchor; static precedes dynamic

6 Re-stringing the Garland

Re-stringing turns pearls back into a prompt. We require three properties, in order:

(i) **Identity.** With no edits, the output equals the input byte for byte. Each pearl stores its trivia, so this holds by construction and is asserted on every run. It is the baseline that makes "works like before" precise.

(ii) **Dependency safety.** A pearl is placed after every pearl it depends on and stays in its anchor's scope.

(iii) **Obligation preservation.** The set of *must* and *never* statements is unchanged, modulo approved rewrites.

Algorithm 1 reorders for caching while keeping original order as the tie-breaker. Stable ordering is deliberate: absent a reason to move, a pearl stays where it was, which limits unintended behavioural drift.

Fused pearls. Some pearls mix static and dynamic content, for example "You are a helpful banking assistant for `{{customer_name}}`." The role is the garland's clasp and must stay first, but its variable ends the cacheable prefix. The checker reports a static-before-dynamic failure and flags the pearl for a *split*. The split is an LLM `rewrite` operation producing a static role plus a dynamic context line, which must then pass the checks again.

Recovering a lost thread. When only loose segments exist, for example from a legacy split with no spans, the thread can be estimated:

- cluster by embedding to recover sections;
- ask the LLM which of two pearls should come first, for pairs within a cluster, and aggregate those judgments into an order;
- treat anaphora as fixed links.

This is slower and less certain than capturing the thread at parse time, which is why PIR stores spans and trivia from the start.

7 Refactoring Conditional Templates

A Jinja prompt is a program whose outputs are prompt *variants*. Flags are few, so the variant set is small enough to enumerate. The domain is inferred from the guards (booleans; compared string constants plus an "other" value) and narrowed with known configuration, such as a flag that is always false in production. Table 6 lists the passes.

Table 6: Refactoring passes over guarded pearls, with their code-refactoring counterparts.

Pass	Code analogue	Rule
Dedupe	Common subexpression	Identical text under several guards becomes one pearl whose guard is the disjunction.
Hoist	Loop-invariant code motion	A pearl printed in every variant becomes unconditional.
Dead / unwrap	Dead-code elimination	A guard false in every variant is deleted; one true in every variant is unwrapped.
Group	Consolidate conditional	All pearls with the same guard in a section form one `{% if %}` block.
If/else	Consolidate duplicate conditional fragments	Blocks guarded by g and $\neg g$ become one `if/else`.
Order	Code motion	Unconditional static text, then conditional blocks (most stable flags first), then dynamic `{{vars}}` in a final CONTEXT section.
Specialise	Replace conditional with polymorphism	Whole blocks guarded by channel or intent leave the template and become code-selected packs, each with its own cache key.
Generalise	Extract method / parameterise	A *pattern family*, the same sentence frame repeated with different objects, becomes one general rule (proposed by the LLM, validated by evaluation).

Translation validation. Rather than proving the passes correct in general, we validate each output [25]. For every variant we render the old and new templates and compare their sets of statements (list markers and headings ignored). Approved rewrites are applied to the old side first, so the comparison is "modulo the rewrite". We also report total words and the *cacheable prefix*, the text before the first per-request value, together with the number of distinct prefixes.

What validation does and does not prove. Equality of statement sets proves that no statement was lost or added in any variant. It does not prove identical model behaviour: position effects are real [1]. The final gate is therefore behavioural, running the golden evaluation set per variant. Set equality has one blind spot: a statement lost from one place survives the check if the same text also prints elsewhere in that variant. An *emitted-exactly-once* check closes it by requiring every kept pearl to appear exactly once in the output. On the 50-page prompt of §12 this check caught a real bug that set equality had passed: three anaphoric pearls whose anchor had been subsumed as a duplicate were dropped, until dependency chains were made to travel with their root.

8 Block Stack: Divide, Optimise, Merge

Re-stringing and template refactoring treat the prompt as one unit. Real prompts are too large for one LLM pass to optimise well, and one failed edit should not force a rollback of everything. We therefore break the prompt into *blocks* that can be moved and optimised independently, then merge them back recursively. Figure 2 shows each stage as one row, drawn from a real run.

What makes a block independent. A set of pearls is a movable block when two conditions hold:

(i) it shares one guard and one cache tier (role, static, per tenant, per session, per request);

(ii) no dependency edge leaves it.

Level 1 partitions by (guard, tier). Level 2 applies union-find over the dependency edges inside each group, welding an anaphoric pearl to its anchor and an exception to its rule. An edge that crosses into a different guard does not weld blocks. It becomes an ordering constraint, so blocks stay small and guards stay separate.

Optimise the leaves. Each atomic block becomes one LLM task, and the tasks run in parallel. Small tasks are cheaper and are judged more reliably than one long prompt. Blocks are independent in text but not in meaning, because the model reads the whole prompt. Each task therefore receives the global must/never rules as a read-only interface, like a header file. It may rewrite only its own pearls and must not contradict that interface.

Merge bottom-up. At every internal node, merging takes three steps:

(i) **Subsumption.** A guarded pearl whose text also prints unconditionally adds nothing in any variant, so it is removed. This catches redundancy *between* blocks, which no leaf task can see.

(ii) **Ordering.** Children are ordered by cache tier, subject to the dependency constraints, with original position as the tie-breaker.

(iii) **Validation.** The node is checked for obligations and dependency safety. At the root, translation validation (§7) runs over every variant.

Rollback is per block: if one block's rewrite fails its checks or the evaluation gate, only that block reverts.

9 The Prompt Stack: Lessons from the OSI Model

The block stack still has to decide where each block goes. Networking solved a similar problem decades ago. The OSI reference model separates concerns into layers with fixed interfaces [17, 18]. On the sending side, each layer *encapsulates* the payload from the layer above by adding its own header, so a frame on the wire reads outermost header first and application data last. We borrow the principles rather than the seven layers themselves. Table 7 lists what we take and what each buys.

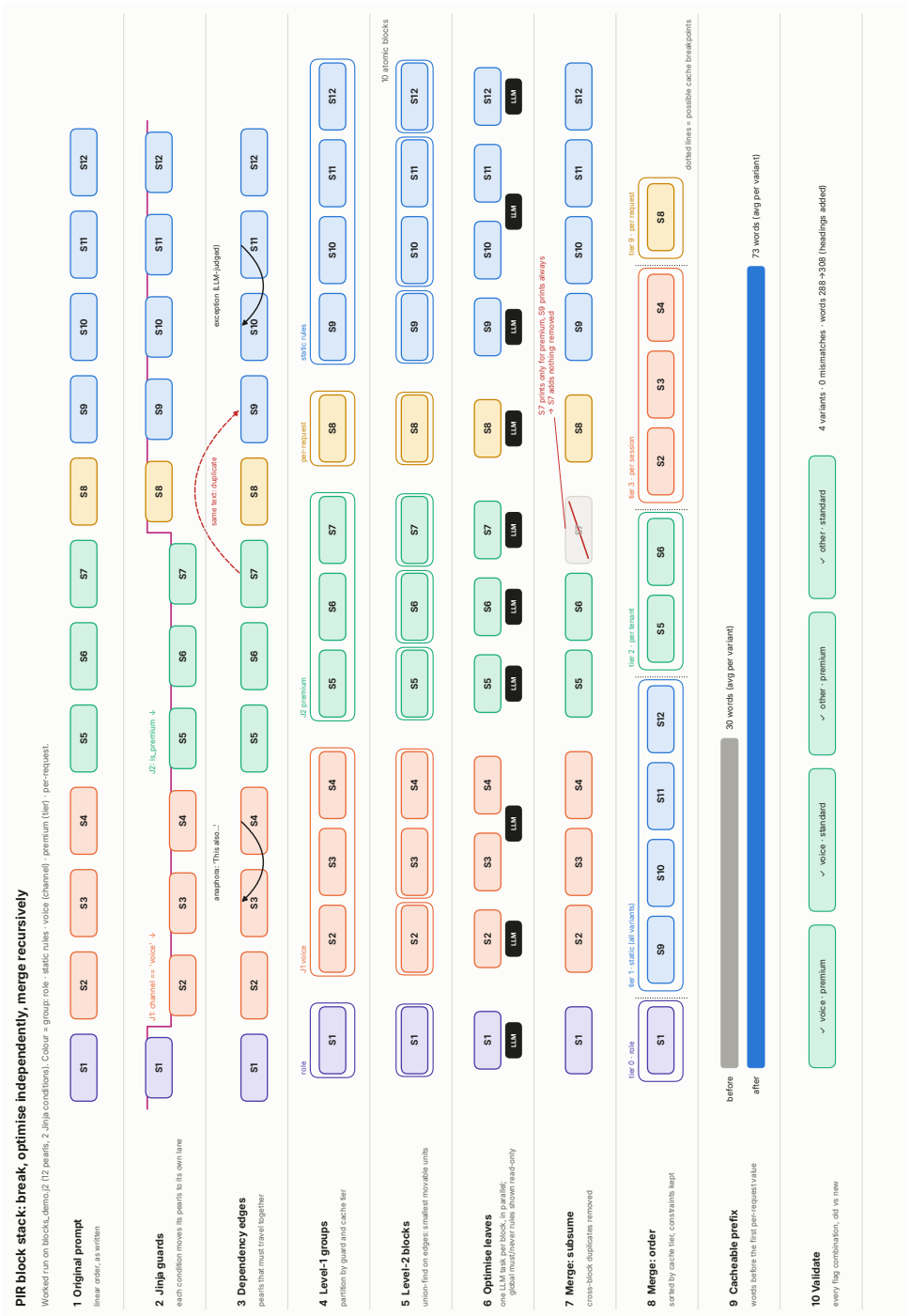


Figure 2: The block stack on the worked template (12 pearls, two Jinja conditions). Rows 1–3: pearls, guard lanes and dependency edges. Rows 4–5: level-1 groups and level-2 atomic blocks (10). Row 6: one LLM task per block, with global rules read-only. Rows 7–8: merge by subsumption (S7 duplicates S9) and by cache tier, where dotted lines mark candidate cache breakpoints. Rows 9–10: cacheable prefix 30 → 73 words per variant, and 0 mismatches over 4 variants.

Table 7: What the Prompt Stack borrows from the OSI model.

OSI principle	Prompt Stack analogue	What it buys
Layers with one concern each	Six concern layers: Identity, Policy, Capabilities, Procedures, Presentation, Context	Every pearl has one home; a "layer violation" becomes detectable.
Encapsulation, outer headers first	Envelopes by scope: global ⊃ tenant ⊃ session ⊃ request payload	Reading order equals stability order, so the cache prefix is as long as possible.
Standard header layout	The same layer order inside every envelope	Ordering becomes a lookup, not a search.
Layer n uses only layer $n-1$'s service	Dependencies may point only outward (inner → outer)	Moving or changing an inner envelope can never break an outer one.
Peer-to-peer protocols, conformance tests	One probe suite per layer (policy refusals, tool calls, format checks)	A layer can be changed when its own suite passes.
MTU and fragmentation	A word or token budget per envelope; overflow is fragmented into on-demand skills	Fragmentation is used only for rarely needed procedures, because every fragment costs a round trip.
Header overhead	Section headings cost words in every variant	An explicit trade-off between readability and length (Table 8).
Protocol versioning per layer	Cache key composed from envelope versions	One envelope can be updated without invalidating the others' cache.

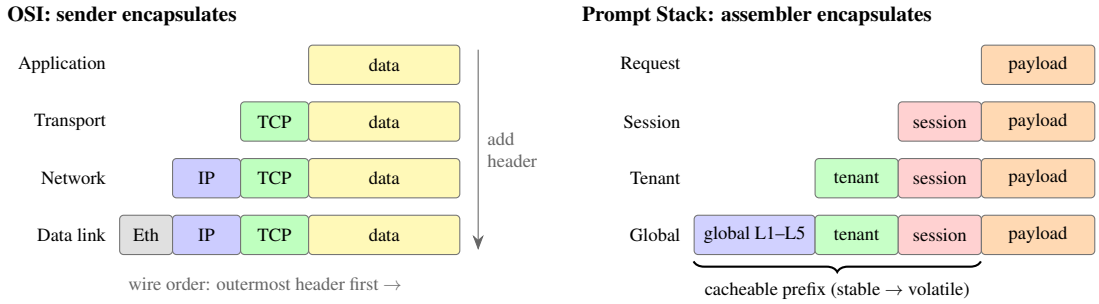


Figure 3: Encapsulation in networking and in prompts. The request payload (the per-request `{{variables}}`) is wrapped by the session envelope, then the tenant envelope, then the global envelope. The model reads outermost first, so the most stable text comes first and the cacheable prefix ends only where the per-request payload begins.

Two axes. Every pearl receives a *layer*, which says what it is about, and a *scope*, which says when it changes.

- **Layer** comes from the kind taxonomy plus lexical cues. Tool mentions go to Capabilities, "never"-type rules to Policy, and output-shape rules to Presentation. A dependant inherits its anchor's layer.
- **Scope** comes from the guard and variables: unguarded pearls are global, guards over tier flags are tenant, guards over channel flags are session, and `{{variables}}` are request. The role pearl is pinned to the global envelope.

Figure 4 places the worked example on this grid.

Emission is a lookup. The stacked prompt is produced in a fixed order:

- envelopes outer to inner;
- inside each envelope, layers L1 to L6;
- one `{% if %}` per guard, with its heading inside the condition so other variants never see it.

Unlike the block stack's merge, nothing is searched.

Lint rules. Borrowing networking's vocabulary gives concrete, automatic checks:

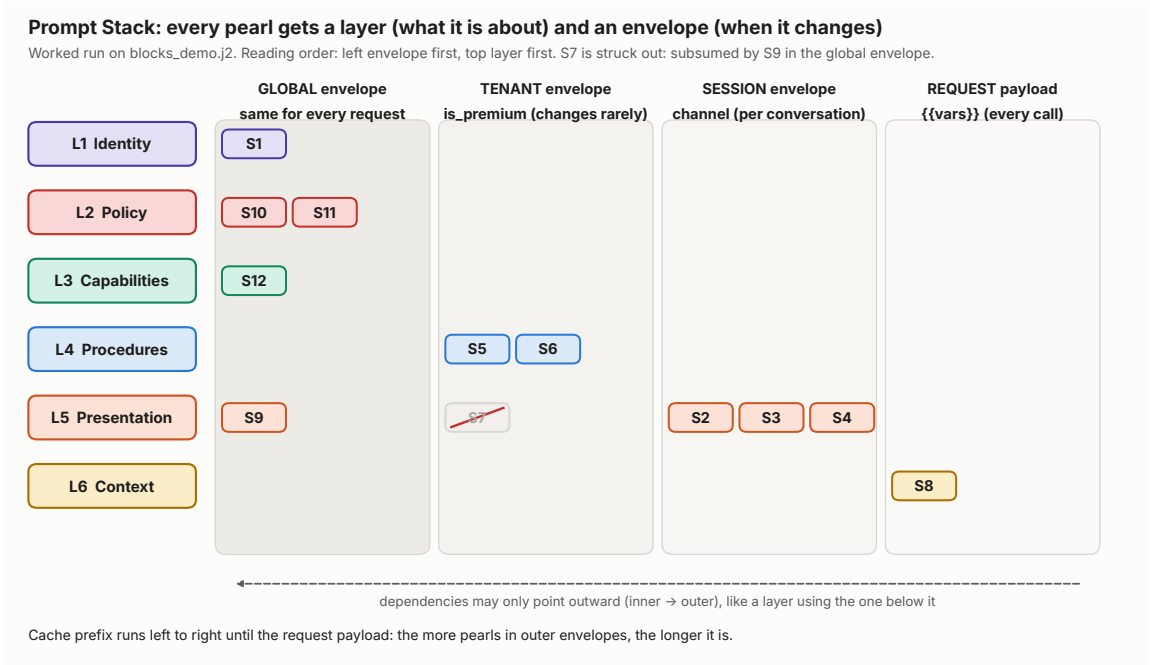


Figure 4: The worked example on the layer × envelope grid. Colours are concern layers; columns are envelopes, read left to right. S7 (Presentation, tenant) is subsumed by S9 (Presentation, global). Dependencies point outward only.

- **Layer violation**: a per-request variable outside the request envelope. On the earlier template this flags "You are a banking assistant for `{{customer_name}}`", the fused pearl of §6.
- **Inward dependency**: an outer-envelope pearl that leans on an inner one.
- **Policy in a variable envelope**: a "never" rule that applies to only some variants, which needs confirmation.
- **MTU exceeded**: an envelope over its budget. On the earlier template the session envelope used 30 words against a 15-word test budget.

Frame anatomy and header overhead. Figure 5 draws each variant as a frame. Before stacking, the per-request payload sits mid-frame; after, it is the innermost and last segment. Table 8 shows the cost of headings, which act as headers. With layer headings, the prompt is longer but labelled. Without them, it is *shorter* than the original, and the cacheable prefix still doubles. Both outputs pass translation validation with zero mismatches.

Table 8: Header overhead on the worked template (4 variants, totals over all variants; prefix is the average per variant).

Output	Rendered words	Cacheable prefix (words)	Mismatches
Original template	288	30	–
Block stack (§8), section headings	308	73	0
Prompt Stack, layer headings	324	77	0
Prompt Stack, no headings	**260** (−9.7%)	61	0

Why this makes the problem easier.

- **Order is settled before optimisation.** The block stack must *decide* an order at every merge; the Prompt Stack fixes it. Recursive merging reduces to subsumption and validation inside each grid cell.

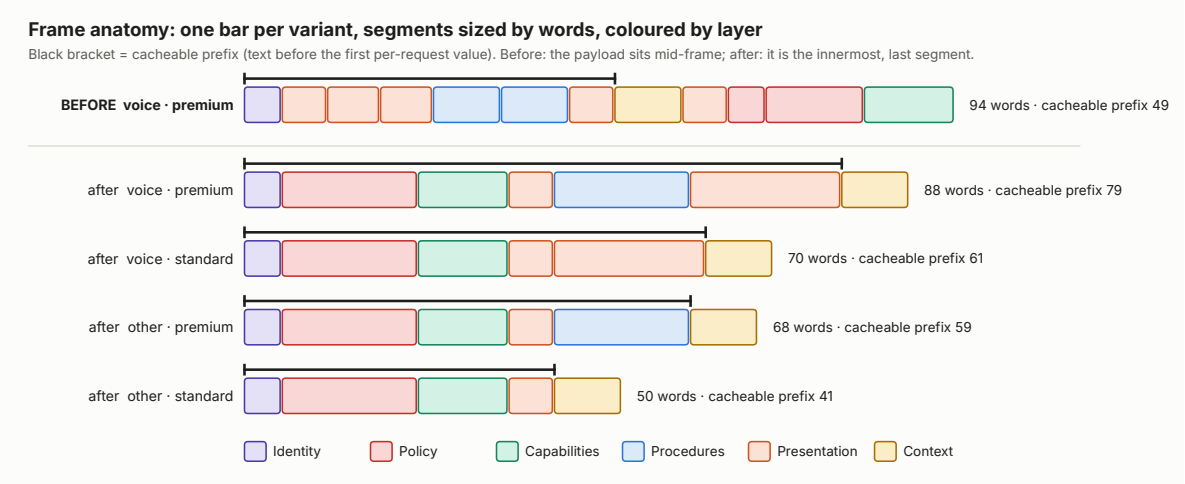


Figure 5: Frame anatomy per variant (pearl words only, headings excluded). Brackets mark the cacheable prefix. For the voice · premium variant, it grows from 49 to 79 words.

- **Each cell is a natural unit** for an LLM optimisation task, an owner and a probe suite. Compliance owns Policy, the platform team owns Capabilities, product owns Procedures.
- **The envelope versions compose the prompt-cache key**, for example `global@5bbfd5|variant@dc728e` in the run, so a tenant change never invalidates the global prefix.

10 Separation of Concerns: Splitting and Merging Agents

The Prompt Stack separates concerns *inside* one prompt: each pearl has a layer, which says what it is about, and an envelope, which says when it changes. Agent design asks the same question one level up: which concerns belong in one agent? A prompt that serves several intents carries every intent's procedures and tool schemas on every call. Several agents that grew separately carry copies of the same identity and policy, and drift apart. PIR already holds what is needed to decide: layers, envelopes, intent membership from the semantic layer (§5), and dependency edges. Splitting and merging are therefore two more passes over the same representation, with the same discipline: deterministic code proposes the structure, the LLM judges only candidates, and every result is checked.

One rule per layer. Table 9 gives the rule. Concerns that every agent needs are *copied* on a split and *subsumed* on a merge. Concerns that belong to one intent are *partitioned* on a split and *guarded* on a merge. This mirrors Parnas's criterion [27]: the cut follows what changes together (an intent's procedures and tools), not the order in which the prompt was written.

Splitting. Algorithm 2 is the block stack's level 2 with intents in place of guards. Pearls are welded into units by anaphora and exception edges, units are grouped by intent, and each bipartition of intents is scored by the quantity that matters at run time: expected static input per request,

$$E_{\text{split}} = R + \sum_i m_i S\big(a(i)\big), \qquad E_{\text{one}} = S_{\text{all}}, \tag{1}$$

where m_i is the share of requests with intent i, $a(i)$ is the agent that serves it, $S(\cdot)$ is an agent's static prompt plus tool schemas, and R is the router. Copies are not penalised separately: a copied anchor or tool schema simply makes S larger. Each agent is emitted with the Prompt Stack, so the global envelope is byte-identical across agents and their cacheable prefixes share one entry.

Table 9: How each concern layer is handled when a prompt is split into agents and when agents are merged.

Layer	Split one prompt into agents	Merge agents into one
L1 Identity	Copied into every agent (name may be specialised)	Aligned; one role in the global envelope
L2 Policy, unconditional	Copied into every agent, byte-identical	Duplicates subsumed into the global envelope
L2 Policy, tool- or procedure-bound	Travels with its tool or anchor (exception edges weld it)	Stays inside its agent's guarded block
L3 Capabilities	Partitioned by intent; a tool needed across the cut is copied with its schema	Union of tools; shared tools stated once
L4 Procedures	Partitioned by intent	One guarded block per agent: `{% if agent == ... %}`
L5 Presentation	Copied	Aligned; differing values are *conflicts*, flagged for a human
L6 Context	Request payload, unchanged	Request payload, unchanged
Edges	A depends-on edge across the cut: copy its anchor, or make it a handoff contract	Edges inside each guard are kept; none may cross guards

Algorithm 2 Splitting a prompt into two agents

Require: PIR pearls with layer, envelope, tools, deps; intent per procedure pearl; traffic mix m

shared $\leftarrow$ L1, L5, L6 and unconditional L2 pearls ▷ copied into every agent
$U \leftarrow$ union-find over anaphora and exception edges among the other pearls ▷ units
$G \leftarrow$ units grouped by intent
for each bipartition (G_A, G_B) of G **do**
    close each side over depends-on edges: copy anchors and their tool schemas
    $E \leftarrow R + \sum_i m_i S(\text{side serving } i)$ ▷ Equation 1
end for
keep the bipartition with the smallest E; emit each side as a Prompt Stack
check: coverage; policy replicated; dependency closure; $E < E_{\text{one}}$

Crossing edges. A depends-on edge that crosses the cut has two resolutions. In a *router* split, where agents are alternatives chosen per request, the anchor is copied into the other agent with any tool it needs. In a *pipeline* split, where one agent's output feeds the next, the edge becomes the *handoff contract*: the variables the anchor produces (a booking reference, a chosen action) must be passed explicitly, and the receiving agent's prompt names them in its request envelope.

Checks. Three structural checks gate a split. *Coverage*: every original pearl appears in at least one agent. *Policy replication*: every unconditional policy pearl appears in every agent, and every tool-bound policy appears wherever its tool does, so a split can never drop a "never" rule from an agent that can act. *Dependency closure*: every pearl's anchors are present in its agent or arrive through the handoff contract. As with templates, these checks prove structure, not behaviour; routing accuracy and context lost at a handoff need the evaluation gate.

Merging. Merging runs the same table in reverse. Both agents are parsed into PIR and their pearls aligned: identical text is subsumed at once; embeddings propose same-topic pairs across agents, and the LLM judges each as duplicate, exception, compatible or conflicting (Table 3). Duplicates are subsumed into the global envelope. Conflicts, such as two different word limits or opposite formatting rules, are never resolved automatically; they are surfaced for an owner. Each agent's remaining pearls become one guarded block, and the result is emitted with the Prompt Stack. Translation validation then runs per mode: rendering the merged template with `agent` set to each original agent must reproduce that agent's pearl set, modulo approved merges. If the merged prompt exceeds an envelope budget, rarely used procedures are fragmented into on-demand skills (Table 7).

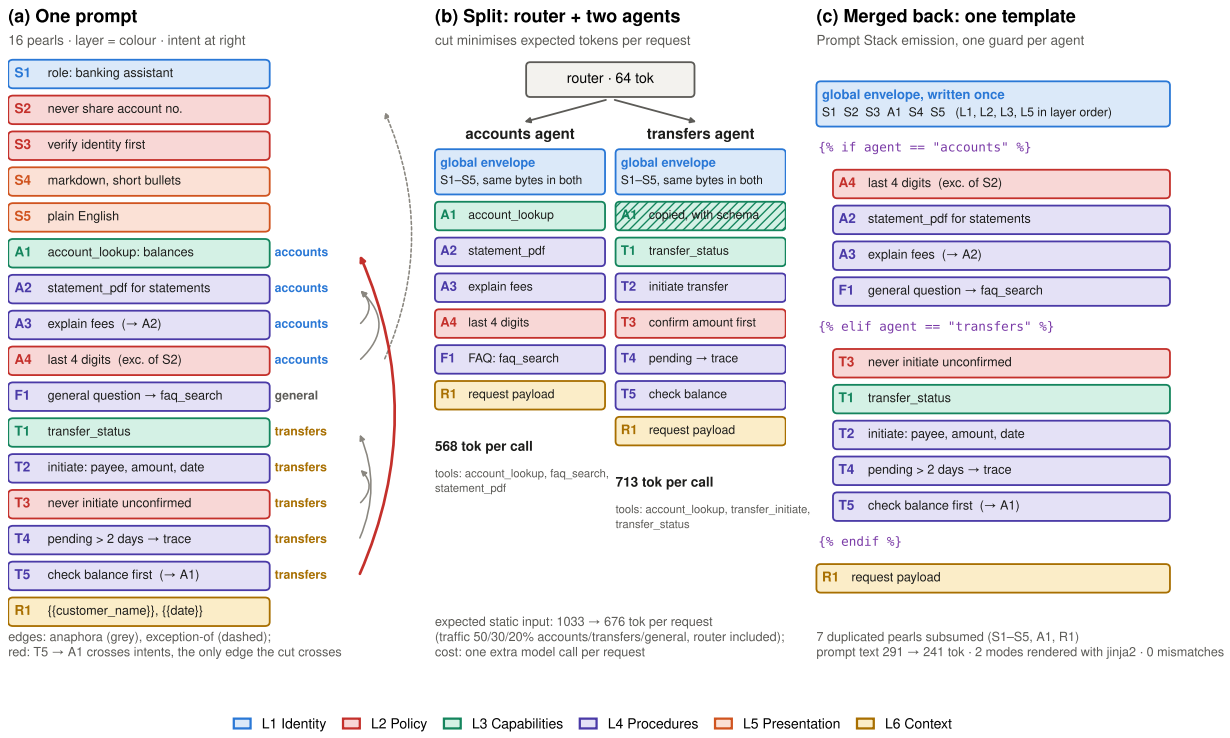


Figure 6: Separation of concerns across agents on a 16-pearl banking prompt. (a) The prompt, coloured by layer, with intent per procedure pearl and dependency edges; one edge (T5 → A1) crosses intents. (b) The cut that minimises expected static input: accounts and general questions in one agent, transfers in the other, A1 copied with its tool schema, and an identical global envelope in both. (c) Merging the two agents back: shared pearls written once, each agent's pearls under one guard, validated per mode with jinja2. Tool schema sizes and the traffic mix are assumptions; all other numbers are computed by `pir_agents.py`.

Worked example. Figure 6 and Table 10 apply both operations to a 16-pearl banking prompt with three intents (accounts, transfers, general questions) and five tools. Intent membership was assigned by hand, as the intent pass would; tool schema sizes (110 to 260 tokens) and a 50/30/20% traffic mix are assumptions, and tokens are estimated at four characters per token. Of the three possible cuts, the one that keeps accounts with general questions and moves transfers out has the lowest expected input. Its only crossing edge, "check the available balance with account_lookup", copies A1 and its schema into the transfers agent. The split lowers expected static input from 1,033 to 676 tokens per request, router included, but adds one model call per request. Merging the two agents back writes seven shared pearls once and passes validation in both modes.

When to split and when to merge. Equation 1 prices tokens; each extra agent also costs a hop, and in a multi-tier chat system hops, not tokens, often dominate latency [28]. A router built from the intent centroids of the semantic layer, rather than an LLM call, keeps that hop cheap. As a rule of thumb, split when intents have disjoint procedures and tools, the cut crosses few edges, E_{split} is well below E_{one}, or the concerns have different owners and release cadences. Merge when two agents share most of their pearls, when traffic often hands off between them (two hops per request), or when the agents are small enough that the router dominates. The Context Tape [31] shows the effect per agent: a split narrows each agent's tool-definitions band, and a merge removes a lane. The public corpus already shows the merge pattern at small scale: smolagents' three agent files repeat 42 statements (§12, Finding 3), which a merge-style subsumption would move into one shared global pack without merging the agents themselves.

11 Case Study

We applied the prototype to a small banking assistant in four forms: a plain prompt embedded in Python, the same prompt with an anaphoric rule, a Jinja template whose `channel`, `is_premium` and `legacy_mode` conditions are inter-

Table 10: Worked example: split and merge of a 16-pearl banking prompt (computed by `pir_agents.py`).

Step	Result
Input	16 pearls: 6 shared (identity, two policies, two presentation, request payload), 10 in 6 units over 3 intents; 5 tools; static input 1,033 tokens (223 text, 810 tool schemas).
Candidate cuts, E_{split}	{accounts, general} | {transfers}: **676**; {accounts} | {general, transfers}: 705; {general} | {accounts, transfers}: 825 (tokens per request, router of 64 tokens included).
Chosen agents	Accounts: 11 pearls, 3 tools, 568 tokens per call. Transfers: 12 pearls, 3 tools, 713 tokens per call.
Crossing edge	T5 $\rightarrow$ A1; A1 and the `account_lookup` schema copied (195 tokens).
Checks	Coverage, policy replication and dependency closure all pass.
Trade-off	-35% expected static input per request; $+1$ model call (router) per request.
Merge back	7 pearls subsumed; prompt text 291 $\rightarrow$ 241 tokens; 2 modes rendered with jinja2; 0 mismatches.
Merging separately written agents	A second team's "Do not reveal account numbers to the customer" scores TF-IDF cosine 0.25 against S2 (a paraphrase), and "Respond in plain text without markdown" scores 0.38 against S4 (a conflict). The lexical stand-in ranks the conflict above the paraphrase; dense embeddings plus the LLM judge are needed, and are not evaluated here.

leaved line by line, and a 12-pearl template shaped like Figure 2 for the block and Prompt Stack runs. Table 11 summarises the results.

The example is deliberately small, and two caveats apply. First, the gains here are structural: readability, grouped conditions and a cacheable prefix. Length did not fall: the refactored template is slightly longer. Large real prompts with heavy duplication should shrink, but we have not yet measured that. Second, the semantic layer was exercised with a lexical TF-IDF stand-in for embeddings, which proves the plumbing only. It found the scattered transfer rules and the account-number conflict. It missed a paraphrased duplicate ("Always answer in plain English" vs. "Always respond using plain and simple English") that dense embeddings are expected to catch.

12 Evaluation on Public Prompt Templates

The earlier sections use small hand-made examples. To test the passes on prompts written by others, we ran the same analysis as the explorer (§13), ported to a script (`pir_scan.py`), on 19 Jinja prompt templates from four open-source agent projects:

- **OpenHands** 0.59.0 [19]: CodeAct and read-only agents, SWE-bench prompts;
- **smolagents** [20]: code, structured-code and tool-calling agents, system and planning prompts;
- **goose** [21]: system and sub-agent prompts;
- **SWE-agent** [22]: default instance template.

Includes are inlined, and loops are kept and treated as printing. Tokens are estimated at four characters per token. Commit hashes are listed in Table 14.

Finding 1: validation holds on real templates. Translation validation found zero mismatches across 2,353 variants. The OpenHands `additional_info` template alone has ten `if`-blocks and 2,048 flag combinations, all enumerated.

Finding 2: structure must be recognised before deduplication. A naive first pass, which treated every line as a statement, reported 343 "exact copies" and 229 "removable" segments. Almost all were structure, not repetition:

- XML-style section tags such as `<ROLE>` and `</ROLE>`;
- code fences;
- variable-only lines such as `{{ code_block_opening_tag }}`;
- lines of example transcripts.

Table 11: Worked-example results (prototype, October 2026). The template run assumes `legacy_mode` is always false, as supplied from configuration.

Check	Result
Identity round trip	Byte-identical rebuild of the original prompt.
Classification	9 pearls: 1 role, 2 instruction, 1 prohibition, 2 condition, 1 format, 2 context (S2 and S6, sent to the LLM).
Conflict detection	"Never share account numbers" vs. "Include account numbers when the user asks for a statement"; to be judged as an exception.
Cache hazard	Per-request variables in the first two pearls; static-before-dynamic check fails until the role pearl is split.
Re-stringing	"Otherwise…" kept with its anchor; the exception placed beside its rule; `{{date}}` moved last; all obligations kept; no dangling references.
Template variants	4 (`channel` $\in$ {voice, other} $\times$ `is_premium` $\in$ {T, F}).
Passes applied	2 dedupes, 2 hoists, 1 dead branch removed, 3 scattered conditional lines grouped into 2 blocks, 1 if/else consolidation.
Translation validation	0 mismatches across all variants (structural passes alone, and with the approved role split).
Rendered words (all variants)	316 $\rightarrow$ 324 (passes only) $\rightarrow$ 332 (with split): +2.5%, from a new section heading and one context sentence.
Cacheable prefix	8 $\rightarrow$ 79 words per variant after the approved role split; distinct prefixes 1 $\rightarrow$ 4, one per channel/tier variant.
Block stack (12-pearl template)	5 level-1 groups, 10 atomic blocks, 10 parallel LLM tasks; S7 subsumed; 0 mismatches over 4 variants; prefix 30 $\rightarrow$ 73 words.
Prompt Stack (same template)	Every pearl placed on the layer $\times$ envelope grid; lint clean; 0 mismatches; 288 $\rightarrow$ 260 words without headings; prefix 30 $\rightarrow$ 61 (77 with headings).
Prompt Stack lint (earlier template)	Layer violation on the role pearl carrying `{{customer_name}}`; session envelope over its test budget.
Split and merge (16-pearl prompt)	Cut chosen from 3 candidates; expected static input 1,033 $\rightarrow$ 676 tokens per request with one extra call; merge back with 7 pearls subsumed and 0 mismatches (§10).

Removing any of these would have broken the prompt. Table 15 shows how each structural rule removed false positives. All of them are now part of the parser, the script and the explorer.

Finding 3: curated prompts are clean inside each file; duplication lives between files. After the structural rules, no template contained a removable duplicate. Copies did appear *across* files: 46 statements and about 2,000 tokens. Most (42 statements, about 1,900 tokens) sit in smolagents' three agent YAML files, which repeat the same opening and planning text. A further four sit in OpenHands' two copied `additional_info` templates. This is exactly the reuse the block stack and Prompt Stack target: one shared pack, included by each agent (§10).

Finding 4: per-request values often come first. Benchmark and instance templates put the task text at the top. Moving per-request values to the innermost envelope gives these average cacheable prefixes:

Template	Prefix before	Prefix after (est. tokens)
OpenHands SWE-bench `swe_default`	40	845
SWE-agent `instance_template`	0	210
OpenHands CodeAct `additional_info` (average of 2,048 variants)	3.8	43.0
smolagents tool-calling `system_prompt`	726	762

Finding 5: pattern families exist in real prompts too. Two families survived the structural rules. One is the SWE-bench phase list ("5.1 State clearly what the problem is", "5.2 State clearly where the problem is located", and so on): a single sentence frame repeated with a different object. Such families can often be refactored into one general rule.

Contrast with the synthetic 50-page sample. The explorer's built-in synthetic sample is a generated banking prompt of 586 segments, about 9,700 tokens, 61 `if`-blocks and 9 flags. On it the analysis reports:

- 87 exact copies and 21 pattern families;
- 78 removable segments;
- for one variant, about 9% fewer tokens and a cacheable prefix growing from 16 to about 8,400 tokens;
- 0 mismatches over all 1,024 variants.

These numbers come from a generator that deliberately injects copies, so they should not be read as typical. The public corpus marks the clean end of the range, and enterprise prompts that grow by accretion are likely to fall between the two. Measuring that requires running the scanner on production prompts and reporting only aggregates.

A labelled 50-page prompt. For v2.3 we wrote a second generator (`gen_prompt.py`) whose output reads like a prompt that has grown by accretion, and which records a gold label for every statement: 50 pages (22,812 words), 2,126 statements, 176 template tags and ten free flags giving 1,536 variants, across 16 intent sections. It injects copied and reworded global rules, lines repeated under both branches of a condition, a dead `legacy_mode` branch, per-request values near the top, pattern families, contradictions, anaphoric chains, four terms defined once in one section and used throughout, and the ambiguous phrasings of §5. Running the passes (`pir_scale.py`) gives the results in Table 12. Two caveats apply. The savings reflect how much copying the generator injects. And the lexical stand-in for embeddings found none of the six planted paraphrases (cosine 0.03–0.64), which confirms that this step needs dense embeddings.

Table 12: PIR passes on the labelled 50-page synthetic prompt (`pir_scale.py`).

Pass or check	Result
Automatic, no LLM	32 dead statements removed; 267 duplicate copies subsumed in 174 groups (3,638 tokens); 16 hoisted.
Tokens per variant	31,298 $\rightarrow$ 28,077 (-10.3%).
Cacheable prefix per request	18 $\rightarrow$ 28,031 tokens on average.
Translation validation	1,536 variants, 0 mismatches; every kept pearl emitted exactly once.
Work left for the LLM	107 atomic blocks of about 262 tokens each; 82 pattern families covering 1,421 statements; 57 conflict candidates.
Intent packs (one typical variant)	26,985 statement tokens for the whole prompt vs. 2,627 for a section pack and 2,206 for a slice, averaged over 16 intents (§12); each pack costs a router call (§10).

The prompt property graph. Code refactoring tools do not work on the syntax tree alone. A safe *extract method* or *rename* also needs def-use chains and call graphs, and the *code property graph* combines the abstract syntax tree, control flow and data dependence for exactly this kind of analysis [29]. PIR's analogue (`pir_graph.py`) keeps the template and markdown trees as the syntactic layer: sections, guards and step order. It adds data edges from every statement that uses a tool to that tool's definition and from every statement that uses a defined term to the term's definition. Semantic edges (anaphora, exceptions, duplicates) come from §5. Definitions are found by a lexical cue on context statements ("A *term* means / has / must be ..."). On the 50-page prompt the graph has 1,827 kept statements (137 guarded) and 299 edges: 220 uses of four defined terms, 30 tool uses and 49 anaphora. The four definitions ("verified identity", "reference number", "priority request", "service hours") were planted by the generator, each in one section and used in all 16; the cue found exactly these four.

The graph supports three analyses that sections cannot. *Slicing* [30]: an intent pack is the backward closure of the intent's own statements plus the always-present global concerns (identity, policy, format, request payload), so it contains what the intent needs and nothing else. *Def-use checks*: a pack in which a statement uses a term or tool whose definition is absent has a dangling dependency. *Impact analysis*: the forward closure of a statement lists everything that changes with it, and therefore which variants and evaluation checks to rerun. Table 13 gives the results. Section packs, built from an intent's section plus all global text, leave 208 uses of terms defined elsewhere without their definition: on average 13 per intent, and in every one of the 16 intents. Slices leave none, and are 16% smaller on average, mainly because they keep only the 2 of 24 tool definitions an intent actually uses. Changing the definition of "verified identity" touches 81 statements in all 16 intents and all 1,536 variants, which a reviewer of that one line would not

see. The edges beyond the template trees are inferred, by cues here and by the LLM with the PIR table in general, so the graph is only as reliable as the tagging; position effects also mean that a structurally complete slice still needs the behavioural gate.

Table 13: Prompt property graph on the 50-page prompt: section-based packs vs. slices, averaged over 16 intents (statement tokens in one typical variant), and one impact query (`pir_graph.py`).

	Section pack	Slice
Tokens per intent (average)	2,627	2,206 (−16%)
Missing dependencies (all 16 intents)	208	0
Intents with a missing dependency	16	0
Tool definitions kept (of 24)	24	2 (average)
Impact of changing the definition of "verified identity": 81 statements, 16 intents, 1,536 variants		

Table 14: Public corpus results per project. "Copied" counts prose statements repeated verbatim across source files of the same project; text shared through `include` counts as reuse, not as a copy. Commits: OpenHands `d39f7ae`, smolagents `96f33fa`, goose `3bd8520`, SWE-agent `3ea751c`.

Project	Templates	Segments	Tokens (est.)	If-blocks	Variants checked	Mismatches	Copied (tokens)
OpenHands 0.59.0	10	710	13,936	20	2,187	0	4 (80)
smolagents	6	379	6,942	9	36	0	42 (1,917)
goose	2	34	605	5	129	0	0
SWE-agent	1	13	283	0	1	0	0
Total	**19**	**1,136**	**21,766**	**34**	**2,353**	**0**	**46 (1,997)**

Table 15: False positives removed by structural rules on the public corpus (within-template counts).

Parser rule added	Exact copies	Removable	Pattern families
Every line is a statement (naive)	343	229	13
+ XML tags are section markers; code blocks are atomic	57	44	10
+ Prose filter ($\geq$4 words, no trailing colon, few code symbols, not variable-only)	4	2	3
+ Example transcripts are never deduplicated	**0**	**0**	**2**

13 Visual Analysis at Scale

A 50-page prompt has hundreds of segments, too many for the pearl diagrams of earlier sections. The companion *Prompt Anatomy Explorer* draws every segment as one tile on a canvas, with width proportional to tokens, and animates the tiles between views. Figure 7 shows two views of the synthetic sample.

- **Document**: reading order, like a page thumbnail, with section names.
- **Condition lanes**: one lane per flag, with x the token offset. It generalises the hand-drawn step diagram that motivated the block stack and shows how often conditions interleave.
- **Prompt Stack**: tiles move into the four envelopes, and subsumed duplicates drop away.
- **Layer grid**: layers by envelopes, with token totals per cell.

Tiles can be coloured by layer, by condition, or by duplicate class (exact, near, pattern family, removable). Choosing flag values renders one variant, fades the tiles that do not print, and marks where the cacheable prefix ends before and after stacking.

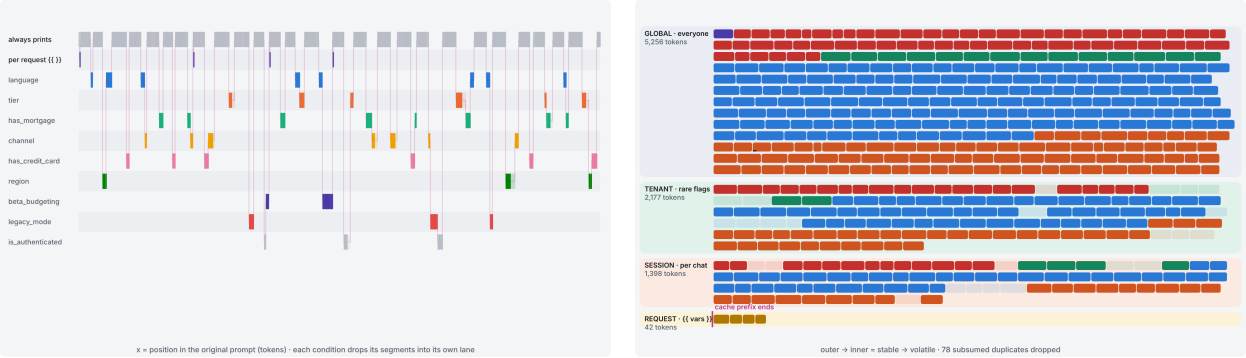


Figure 7: Prompt Anatomy Explorer on the synthetic 50-page sample. Left: condition lanes, coloured by condition, with the step line tracing every branch. Right: the Prompt Stack, coloured by layer, with the global, tenant, session and request envelopes and the per-request payload last.

In-browser execution. Parsing, guard evaluation, duplicate detection, stacking and validation all run in the browser, so a confidential prompt never leaves the analyst's machine. Following the in-browser inference approach of transformers-js [23], an optional pass loads a small sentence-embedding model with Transformers.js [24] to find paraphrases that word overlap misses. This pass needs network access to the model host. It has not been evaluated in this report.

14 Workflow and Integration

The pipeline (Figure 8) is a set of small Python modules, one data file and one browser tool:

- `pir.py`: front end and rule classifier;
- `pir_ml.py`: embeddings and LLM tasks;
- `pir_rebuild.py`: re-stringing and proof checks;
- `pir_jinja.py`: template refactoring and translation validation;
- `pir_blocks.py`: block stack, leaf tasks and recursive merge;
- `pir_layers.py`: Prompt Stack grid, emission, lint and cache keys;
- `pir_agents.py`: splitting a prompt into agents and merging agents, with checks (§10);
- `pir_scan.py`: whole-template scan used in §12, with the same passes as the explorer;
- `pir_table_v1.json`, `pir_rules.py`, `pir_tag.py`: the PIR table, the rule layer with routing, and the tagging harness (§5);
- `gen_prompt.py`, `pir_scale.py`, `pir_graph.py`: the labelled 50-page generator, the end-to-end run and the prompt property graph of §12;
- `prompt_anatomy_explorer.html`: the browser explorer of §13.

Dependencies are `tree-sitter`, `tree-sitter-markdown`, `tree-sitter-python`, `jinja2`, `numpy` and `scikit-learn`, plus an Azure OpenAI client when available.

Copilot in VS Code. In environments where the analysis cannot call an LLM API directly, the scripts act as tools for a Copilot agent. The agent runs the front end, reads the exported task file, judges each task against its JSON schema, writes back operations, and runs the re-stringing and validation steps. The scripts guarantee structure and proofs; the agent supplies judgment. A colour-coded view, as in Figure 1, can be rendered by a small VS Code extension that decorates the byte spans in the PIR JSON.

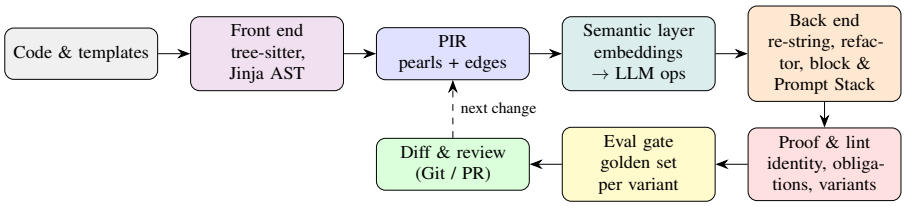


Figure 8: The Prompt Anatomy pipeline. Every change, whether a pass or an LLM operation, passes the proof checks and the evaluation gate before it reaches a pull request.

Place in a remediation programme. PIR implements the "model-agnostic de-bloat" and "intent-scoped context assembly" steps of a broader latency and quality plan for multi-agent chat systems. Its intent packs and static-first ordering feed directly into per-intent prompt-cache keys [6], and its split and merge passes give that plan a principled way to decide agent boundaries.

15 Limitations and Threats to Validity

- **Rule classifier.** The lexical rules are English-specific and heuristic; ambiguous statements depend on the LLM fallback.
- **Equivalence is structural.** Statement-set equality does not imply behavioural equality. Reordering changes attention, and rewrites change wording. The behavioural evaluation gate is mandatory.
- **Enumeration.** Exhaustive variant enumeration is exponential in the number of flags. Beyond roughly ten independent flags, pairwise coverage or a solver over guards is needed.
- **Template coverage.** The Jinja walker supports `if`/`elif`/`else`, comparisons and boolean operators. Loops, macros, includes and filters need extensions; mid-sentence conditions are preserved rather than refactored.
- **The OSI analogy is a design aid, not a theorem.** Concern layers are not strictly hierarchical. Layer and scope assignment is heuristic, and fixed layer order may place a critical rule mid-prompt, so probes and evaluations still decide.
- **Block independence is textual.** Leaf tasks see global rules read-only, but semantic interaction between blocks is caught only by the evaluation gate.
- **Split and merge are shown on one example.** The worked example in §10 is hand-made: intent labels were assigned by hand, and tool schema sizes and the traffic mix are assumed. The cut search enumerates intent bipartitions, which is fine for a handful of intents but needs a graph-partitioning heuristic beyond that. The structural checks do not cover routing errors or context lost at a handoff, and the alignment step was run only with a lexical stand-in.
- **Tagging with the PIR table is not yet measured.** Table 5 scores the rule layer only; the LLM rows must be run on a deployment's own model. The gold labels and ambiguous phrasings are synthetic and were written together with the routing hedges, so routing recall is optimistic.
- **Evaluation scale.** The public corpus is about 22,000 tokens across 19 curated templates, and the 50-page sample is synthetic. A study on production prompts is still needed.
- **Token estimates.** Token counts use four characters per token rather than a model tokenizer.
- **Loops.** Loop bodies are treated as printing once and are not expanded per item.

16 Future Work

- Dense-embedding calibration and a labelled benchmark of duplicate, conflict and exception pairs.
- Measuring the PIR table: thin prompt vs. table on the routed statements, on several models, plus agreement between two runs; then refining the table from its disagreements, versioned like any other artefact.

- A tree-sitter grammar for an explicit PIR notation (e.g. `@role`, `@if ... @then`), giving editor highlighting and round-trippable source.
- Solver-based guard simplification in place of enumeration.
- Automatic generation of *rule probes*: test inputs that trigger each pearl, so every rule has a behavioural test.
- Using PIR as the search space for GEPA or DSPy, with the proof checks as hard constraints [12, 11].
- Order recovery for legacy segments from pairwise LLM judgments.
- Per-layer probe suites and envelope-level budgets learned from production traffic.
- Using GPT-5.6 explicit cache breakpoints at envelope boundaries [6].
- Evaluating the in-browser embedding pass against labelled paraphrase pairs.
- Cross-file analysis that proposes shared packs automatically for agents that copy text.
- Richer graph edges: entities produced by one step and consumed by another, exception and conflict edges from the LLM, and subtree clone detection for pasted blocks; slices as the default unit for intent packs, agent splits and evaluation scope.
- Split and merge on real agent sets (for example smolagents' three agents), with measured traffic mixes, real tool schemas, and the Context Tape's per-agent lanes to compare tokens and hops before and after.

17 Conclusion

Prompts deserve the tools we give code. A typed intermediate representation, with spans, guards and dependency edges, turns prompt maintenance from editing prose into running analyses and refactorings with proofs attached. Embeddings find the candidates and the LLM judges them; deterministic code applies the changes and checks them. The garland can then be re-strung exactly as it was, or better, with nothing required left on the floor. Borrowing encapsulation from networking makes "better" systematic: stable envelopes outside, the volatile payload inside, and each layer owned, tested and versioned on its own. The same separation of concerns decides where one agent should end and another begin.

Version history.

- **v2.4** (October 2026): the prompt property graph (§12): template and markdown trees plus def-use and anaphora edges, used for slicing intent packs, def-use checks and impact analysis; the 50-page generator gains four defined terms, and its numbers are updated throughout.
- **v2.3** (October 2026): the PIR table, a versioned codebook given to the LLM as a cached prefix, with routing from the rule layer (§5); a labelled 50-page synthetic prompt with tagging scores and an end-to-end run (§12); an emitted-exactly-once check that caught a lost-pearl bug which set equality had missed (§7); three new modules and a data file.
- **v2.2** (October 2026): separation of concerns across agents (§10): layer-by-layer rules for splitting and merging, a cut that minimises expected input per request, crossing edges as copies or handoff contracts, per-mode validation of merges, a computed worked example and a new module (`pir_agents.py`). The document was re-typeset from the v2.1 PDF; v2.1 text, tables and figures are unchanged apart from cross-references.
- **v2.1** (October 2026): evaluation on 19 public Jinja templates from OpenHands, smolagents, goose and SWE-agent (§12); structural rules that remove deduplication false positives; pattern families and the "generalise" refactoring; the canvas explorer and in-browser execution (§13).
- **v2.0** (October 2026): adds the block stack (§8) and the OSI-inspired Prompt Stack (§9), four new colour figures, header-overhead measurements, lint rules and two new modules.
- **v1.0** (October 2026): initial release, covering the PIR schema, tree-sitter and Jinja front end, semantic layer, re-stringing with proofs, template refactoring with translation validation, and the worked case study.

References

[1] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 2024. arXiv:2307.03172.

[2] D. Jaroslawicz et al. How Many Instructions Can LLMs Follow at Once? 2025. arXiv:2507.11538.

[3] Y. Du et al. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval. *Findings of EMNLP*, 2025. arXiv:2510.05381.

[4] K. Hong, A. Troynikov, J. Huber. Context Rot. Chroma Research, 2025. https://www.trychroma.com/research/context-rot

[5] OpenAI. Prompt guidance for GPT-5.6. https://developers.openai.com/api/docs/guides/prompt-guidance-gpt-5p6

[6] OpenAI. Prompt caching guide. https://developers.openai.com/api/docs/guides/prompt-caching

[7] E. Lumer et al. Don't Break the Cache. 2026. arXiv:2601.06007.

[8] Y. Ji. Context Engineering for AI Agents: Lessons from Building Manus. 2025. https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus

[9] Anthropic. Effective context engineering for AI agents. 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[10] H. Jiang et al. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. *EMNLP*, 2023. arXiv:2310.05736.

[11] O. Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. 2023. arXiv:2310.03714.

[12] L. A. Agrawal et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. 2025. arXiv:2507.19457.

[13] M. Brunsfeld et al. Tree-sitter: a parser generator tool and an incremental parsing library. https://tree-sitter.github.io/tree-sitter/

[14] Pallets. Jinja template engine documentation. https://jinja.palletsprojects.com/

[15] N. D. Jones, C. K. Gomard, P. Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993.

[16] M. Fowler. *Refactoring: Improving the Design of Existing Code*. 2nd ed., Addison-Wesley, 2018.

[17] H. Zimmermann. OSI Reference Model: The ISO Model of Architecture for Open Systems Interconnection. *IEEE Transactions on Communications*, 28(4):425–432, 1980.

[18] ISO/IEC 7498-1:1994. Information technology: Open Systems Interconnection, Basic Reference Model: The Basic Model.

[19] X. Wang et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. 2024. arXiv:2407.16741. Templates from https://github.com/OpenHands/OpenHands, tag 0.59.0.

[20] Hugging Face. smolagents. https://github.com/huggingface/smolagents

[21] goose, an open-source AI agent. https://github.com/block/goose

[22] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. arXiv:2405.15793. https://github.com/SWE-agent/SWE-agent

[23] F. Krueger. Transformers-js: Neural Networks in the Browser. 2022. https://praeclarum.org/2022/08/25/transformers-js.html

[24] Hugging Face. Transformers.js. https://github.com/huggingface/transformers.js

[25] A. Pnueli, M. Siegel, E. Singerman. Translation Validation. In *TACAS*, LNCS 1384, Springer, 1998.

[26] E. W. Dijkstra. On the role of scientific thought (EWD447), 1974. In *Selected Writings on Computing: A Personal Perspective*, Springer, 1982.

[27] D. L. Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[28] A. R. Sharma. Hops, Not Tokens: latency of a three-tier agent chat solution. Technical report, October 2026.

[29] F. Yamaguchi, N. Golde, D. Arp, K. Rieck. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *IEEE Symposium on Security and Privacy*, 2014.

[30] M. Weiser. Program Slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[31] A. R. Sharma. Context Tape: Observing, Costing and Controlling Context Growth in Multi-Turn LLM Agents. Technical report, v1.3, October 2026.